

Id lead to corruption of repositories in the fork network. Therefore, objects in the pooled repository must continue to exist as long as there are repositories referencing them.

Without a mechanism to accurately determine if a pooled object is referenced by one of more repositories, all objects in the pooled repository must remain. Only when there are no repositories referencing the object pool can the pooled repository, and therefore all its objects, be removed.

Object sharing

An object that is deduplicated will become accessible from all forks of a particular repository, even if it has never been reachable in any of the forks. The consequence is that any write to an object pool immediately influences all of its members.

We need to be mindful of this property when repositories connected to an object pool are replicated. As the user-observable state should be the same on all replicas, we need to ensure that both the repository and its object pool are consistent across the different nodes.

Proposal

In the current design, management of object pools mostly happens on the client side as they need to manage their complete lifecycle themselves. This requires Rails to store the object pool relationships in the Rails database, perform fine-grained management of every single step of an object pool's life, and perform periodic Sidekiq jobs to enforce state by calling idempotent Gitaly RPCs. This design significantly increases complexity of an already-complex mechanism.

Instead of handling the full lifecycle of object pools on the client-side, this document proposes to instead encapsulate the object pool lifecycle management inside of Gitaly. Instead of performing low-level actions to maintain object pools, clients would only need to tell Gitaly about updated relationships between a repository and its object pool.

This brings us multiple advantages:

- The inherent complexity of the lifecycle management is encapsulated in a single place, namely Gitaly.
- Gitaly is in a better position to iterate on the low-level technical design of object pools in case we find a better solution compared to "alternates" in the future.
- We can ensure better interplay between Gitaly Cluster, object pools and repository housekeeping.
- Gitaly becomes the single source of truth for object pool relationships and can thus start to manage it better.

Overall, the goal is to raise the abstraction level so that clients need to worry less about the technical details while Gitaly is in a better position to iterate on them.

Move lifecycle management of pools into Gitaly

The lifecycle management of object pools is leaking too many details to the client, and by doing so makes parts things both hard to understand and inefficient.

The current solution relies on a set of fine-grained RPCs that manage the relationship between repositories and their object pools. Instead, we are aiming for a simplified approach that only exposes the high-level concept of forks to the client. This will happen in the form of three RPCs:

- ForkRepository() will create a fork of a given repository. If the upstream repository does not yet have an object pool, Gitaly will create it. It will then create the new repository and automatically link it to the object pool. The upstream repository will be recorded as primary member of the object pool, the fork will be recorded as a secondary member of the object pool.
- UnforkRepository() will remove a repository from the object pool it is connected to. This will stop deduplication of objects. For the primary object pool member this also means that Gitaly will stop pulling new objects into the object pool.
- GetObjectPool() returns the object pool for a given repository. The pool description will contain information about the pool's primary object pool member as well as all secondary object pool members.

Furthermore, the following changes will be implemented:

- RemoveRepository() will remove the repository from its object pool. If it was the last object pool member, the pool will be removed.
- OptimizeRepository(), when executed on the primary object pool member, will also update and optimize the object pool.
- ReplicateRepository() needs to be aware of object pools and replicate them correctly. Repositories shall be linked to and unlink from object pools as required. While this is a step towards fixing the Praefect world, which may seem redundant given that we plan to deprecate Praefect anyway, this RPC call is also used for other use cases like repository rebalancing.

With these changes, Gitaly will have much tighter control over the lifecycle of object pools. Furthermore, as it starts to track the membership of repositories in object pools it can become the single source of truth for fork networks.

Fix inefficient maintenance of object pools

In order to update object pools, Gitaly performs a fetch of new objects from the primary object pool member into the object pool. This fetch is inefficient as it needs to needlessly negotiate objects that are new in the primary object pool member. But given that objects are deduplicated already in the primary object pool member it means that it should only have objects in its object database that do not yet exist in the object pool. Consequently, we should be able to skip the negotiation completely and instead link all objects into the object pool that exist in the source repository.

In the current design, these objects are kept alive by creating references to the just-fetched objects. If the fetch deleted references or force-updated any references, then it may happen that previously-referenced objects become unreferenced. Gitaly thus creates keep-around references so that they cannot ever be deleted. Furthermore, those references are required in order to properly replicate object pools as the replication is reference-based.

These two things can be solved in different ways:

- We can set the `preciousObjects` repository extension. This will instruct all versions of Git which understand this extension to never delete any objects even if `git-prune(1)` or similar commands were executed. Versions of Git that do not understand this extension would refuse to work in this repository.
- Instead of replicating object pools via `git-fetch(1)`, we can instead replicate them by sending over all objects part of the object database.

Taken together this means that we can stop writing references in object pools altogether. This leads to efficient updates of object pools by simply linking all new objects into place, and it fixes issues we have seen with unbounded growth of references in object pools.

Design and implementation details

Moving lifecycle management of object pools into Gitaly

As stated, the goal is to move the ownership of object pools into Gitaly. Ideally, the concept of object pools should not be exposed to callers at all anymore. Instead, we want to only expose the higher-level concept of networks of repositories that share objects with each other in order to deduplicate them.

The following subsections review the current object pool-based architecture and then propose the new object deduplication network-based architecture.

Object pool-based architecture

Managing the object pool lifecycle in the current architecture requires a plethora of RPC calls and requires a lot of knowledge from the calling side. The following sequence diagram shows a simplified version of the lifecycle of an object pool. It is simplified insofar as we only consider there to be a single object pool member.

mermaid

sequenceDiagram

Rails->>+Gitaly: CreateObjectPool
Gitaly->>+Object Pool: Create
activate Object Pool
Object Pool-->>-Gitaly: Success
Gitaly-->>-Rails: Success

Rails->>+Gitaly: LinkRepositoryToObjectPool
Gitaly->>+Upstream: Link
Upstream-->>-Gitaly: Success
Gitaly-->>-Rails: Success

Rails->>+Gitaly: OptimizeRepository
Gitaly->>+Upstream: Optimize
Upstream-->>-Gitaly: Success
Gitaly-->>-Rails: Success

Rails->>+Gitaly: CreateFork
Gitaly->>+Fork: Create
activate Fork
Fork-->>-Gitaly: Success
Gitaly-->>-Rails: CreateFork

note over Rails, Fork: Fork exists but is not connected to the object pool.

Rails->>+Gitaly: LinkRepositoryToObjectPool
Gitaly->>+Fork: Link
Fork-->>-Gitaly: Success
Gitaly-->>-Rails: Success

note over Rails, Fork: Fork is connected to object pool, but objects are duplicated.

Rails->>+Gitaly: OptimizeRepository
Gitaly->>+Fork: Optimize
Fork-->>-Gitaly: Success
Gitaly-->>-Rails: Success

loop Regularly

note over Rails, Fork: Rails needs to ensure that the object pool is regularly updated.

Rails->>+Gitaly: FetchIntoObjectPool
Gitaly->>+Object Pool: Fetch
Object Pool-->>-Gitaly: Success
Gitaly-->>-Rails: Success

end

alt Disconnect Fork

note over Rails, Fork: Forks can be disconnected to stop deduplicating objects.

```

Rails->>+Gitaly: DisconnectGitAlternates
Gitaly->>+Fork: Disconnect
Fork-->>-Gitaly: Success
Gitaly-->>-Rails: Success
else Delete Fork
  note over Rails, Fork: Or the fork is deleted eventually.

```

```

Rails->>+Gitaly: RemoveRepository
Gitaly->>+Fork: Remove
Fork-->>-Gitaly: Success
deactivate Fork
Gitaly-->>-Rails: Success
end

```

```

Rails->>+Gitaly: DisconnectGitAlternates
Gitaly->>+Upstream: Disconnect
Upstream-->>-Gitaly: Success
Gitaly-->>-Rails: Success

```

```

Rails->>+Gitaly: DeleteObjectPool
Gitaly->>+Object Pool: Remove
Object Pool-->>-Gitaly: Success
deactivate Object Pool
Gitaly-->>-Rails: Success

```

The following steps are involved in creating the object pool:

1. The object pool is created from its upstream repository by calling `CreateObjectPool()`. It contains all the objects that the upstream repository contains at the time of creation.
1. The upstream repository is linked to the object pool by calling `LinkRepositoryToObjectPool()`. Its objects are not automatically deduplicated.
1. Objects in the upstream repository get deduplicated by calling `OptimizeRepository()`.
1. The fork is created by calling `CreateFork()`. This RPC call only takes the upstream repository as input and does not know about the already-created object pool. It thus performs a second full copy of objects.
1. Fork and object pool are linked by calling `LinkRepositoryToObjectPool()`. This writes the `info/alternates` file in the fork so that it becomes aware of the additional object database, but doesn't cause the objects to become deduplicated.
1. Objects in the fork get deduplicated by calling `OptimizeRepository()`.
1. The calling side is now expected to regularly call `FetchIntoObjectPool()` to fetch new objects from the upstream repository into the object pool. Fetched objects are not automatically deduplicated in the upstream repository.
1. The fork can be detached from the object pool in two ways:
 - Explicitly by calling `DisconnectGitAlternates()`, which removes the `info/alternates` file and reduplicates all objects.

- By calling RemoveRepository() to delete the fork altogether.
1. When the object pool is empty, it must be removed by calling DeleteObjectPool().

It is clear that the whole lifecycle management is not well-abstracted and that the clients need to be aware of many of its intricacies. Furthermore, we have multiple sources of truth for object pool memberships that can (and in practice do) diverge.

Object deduplication network-based architecture

The proposed new architecture simplifies this process by completely removing the notion of object pools from the public interface. Instead, Gitaly exposes the high-level notion of "object deduplication networks". Repositories can join these networks with one of two roles:

- Read-write object deduplication network members regularly update the set of objects that are part of the object deduplication network.
- Read-only object deduplication network members are passive members and never update the set of objects that are part of the object deduplication network.

The set of objects that can be deduplicated across members of the object deduplication network thus consists only of objects fetched from the read-write members. All members benefit from the deduplication regardless of their role. Typically:

- The original upstream repository is designated as the read-write member of the object deduplication network.
- Forks are read-only object deduplication network members.

It is valid for object deduplication networks to only have read-only members. In that case the network is not updated with new shared objects, but the existing shared objects remain in use.

Though object pools continue to be the underlying mechanism, the higher level of abstraction would allow us to swap out the mechanism if we ever decide to do so.

While clients of Gitaly need to perform fine-grained lifecycle management of object pools in the object pool-based architecture, the object deduplication network-based architecture only requires them to manage memberships of object deduplication networks. The following diagram shows the equivalent flow to the object pool-based architecture in the object deduplication network-based architecture:

```
mermaid
sequenceDiagram
    Rails->>+Gitaly: CreateFork
    Gitaly->>+Object Pool: Create
    activate Object Pool
    Object Pool -->>-Gitaly: Success
```

```
Gitaly->>+Fork: Create
Fork->>+Object Pool: Join
Object Pool-->>-Fork: Success
Fork-->>-Gitaly: Success
activate Fork
Gitaly-->>-Rails: CreateFork
```

```
loop Regularly
  Rails->>+Gitaly: OptimizeRepository
  Gitaly->>+Fork: Optimize
  Gitaly->>+Object Pool: Optimize
  Object Pool-->>-Gitaly: Success
  Fork-->>-Gitaly: Success
  Gitaly-->>-Rails: Success
end
```

```
alt Disconnect Fork
  Rails->>+Gitaly: RemoveRepositoryFromObjectDeduplicationNetwork
  Gitaly->>+Fork: Disconnect
```

```
  alt Last member
    Gitaly->>+Object Pool: Remove
    Object Pool-->>-Gitaly: Success
  end
```

```
  Fork-->>-Gitaly: Success
  Gitaly-->>-Rails: Success
else Delete Fork
  Rails->>+Gitaly: RemoveRepository
  Gitaly->>+Fork: Remove
```

```
  alt Last member
    Gitaly->>+Object Pool: Remove
    Object Pool-->>-Gitaly: Success
  end
```

```
  Fork-->>-Gitaly: Success
  deactivate Fork
  Gitaly-->>-Rails: Success
end
```

The following major steps are involved:

1. The fork is created, where the request instructs Gitaly to have both upstream and fork repository join an object deduplication network. If the upstream project is part of an object deduplication network already, then the fork joins that object deduplication network. If it isn't, Gitaly creates an object pool and joins the upstream repository as a read-write member and the fork as a read-only member. Objects of the fork are

immediately deduplicated. Gitaly records the membership of both repositories in the object pool.

1. The client regularly calls `OptimizeRepository()` on either the upstream or the fork project, which is something that clients already know to do. The behavior changes depending on the role of the object deduplication network member:
 - When executed on a read-write object deduplication network member, the object pool may be updated based on a set of heuristics. This will pull objects which have been newly created in the read-write object deduplication network member into the object pool so that they are available for all members in the object deduplication network.
 - When executed on a read-only object deduplication network member, the object pool will not be updated so that objects which are only part of the read-only object deduplication network member will not get shared across members. The object pool may still be optimized though as required, for example by repacking objects.
1. Both the upstream and the fork project can leave the object deduplication network by calling `RemoveRepositoryFromObjectNetwork()`. This reduplicates all objects and disconnects the repositories from the object pool. Furthermore, if the repository was a read-write object deduplication network member, Gitaly will stop using it as a source to update the pool.

Alternatively, the fork can be deleted with a call to `RemoveRepository()`.

Both calls update the memberships of the object pool to reflect that repositories have left it. Gitaly deletes the object pool if it has no members left.

With this proposed flow the creation, maintenance, and removal of object pools is handled opaquely inside of Gitaly. In addition to the above, two more supporting RPCs may be provided:

- `AddRepositoryToObjectDeduplicationNetwork()` to let a preexisting repository join into an object deduplication network with a specified role.
- `ListObjectDeduplicationNetworkMembers()` to list all members and their roles of the object deduplication network that a repository is a member of.

Migration to the object deduplication network-based architecture

Migration towards the object deduplication network-based architecture involves a lot of small steps:

1. `CreateFork()` starts automatically linking against preexisting object pools. This allows fast forking and removes the notion of object pools for callers when creating a fork.
1. Introduce `AddRepositoryToObjectDeduplicationNetwork()` and `RemoveRepositoryFromObjectDeduplicationNetwork()`. Deprecate `AddRepositoryToObjectPool()` and `DisconnectGitAlternates()` and migrate Rails to use the new RPCs. The object deduplication network is identified via a repository, so this drops the notion of object pools when handling

memberships.

1. Start recording object deduplication network memberships in `CreateFork()`, `AddRepositoryToObjectDeduplicationNetwork()`, `RemoveRepositoryFromObjectDeduplicationNetwork()` and `RemoveRepository()`. This information empowers Gitaly to take control over the object pool lifecycle.
1. Implement a migration so that we can be sure that Gitaly has an up-to-date view of all members of object pools. A migration is required so that Gitaly can automatically handle the lifecycle of an object pool, which:
 - Enables `OptimizeRepository()` to automatically fetch objects from read-write object pool members.
 - Allows Gitaly to automatically remove empty object pools.
1. Change `OptimizeRepository()` so that it also optimizes object pools connected to the repository, which allows us to deprecate and eventually remove `FetchIntoObjectPool()`.
1. Adapt `RemoveRepositoryFromObjectDeduplicationNetwork()` and `RemoveRepository()` to remove empty object pools.
1. Adapt `CreateFork()` to automatically create object pools, which allows us to remove the `CreateObjectPool()` RPC.
1. Remove the `ObjectPoolService` and the notion of object pools from the Gitaly public API.

This plan is of course subject to change.

Gitaly Cluster concerns

Repository creation

When a repository is forked for the first time, Rails creates an object pool via the `CreateObjectPool()` RPC. This means object pool creation is handled outside of Gitaly. Subsequently, the object pool is linked to upstream and fork repositories. When a repository has its Git alternates file configured to link to another repository, these two repositories must exist on the same physical storage.

The repository and its object pool existing on the same physical storage is particularly important for Praefect because it is dependent of the repository's replication factor. A replication factor is a configuration that controls how many storages the repository is replicated to in the Praefect virtual storage. By default, the replication factor is equal to the number of storages in Praefect. This means that when using the default replication factor, a repository is available on all storages in the cluster. When a custom replication factor is used, the number of replicas can be reduced so that a repository only exists on a subset of storages in Praefect.

Gitaly Cluster persists repositories and their assigned storages in the Praefect PostgreSQL database. The database is updated when new repositories are created on the virtual storage. When a new repository is created, the replication factor specifies how many storages are randomly assigned to the repository. The following scenario outlines how a custom replication factor can be problematic

for object pools:

1. A new repository is created in a Gitaly Cluster that has five storage nodes. The replication factor is set to three. Therefore, three storages are randomly selected and assigned for this new repository in Praefect. For example, the assignments are storages 1, 2, and 3. Storages 4 and 5 do not have a copy of this repository.
1. The repository gets forked for the first time thus requiring an object pool repository be created with the `CreateObjectPool()` RPC. Because the replication factor is set to three, another randomly selected set of three storages are assigned in Praefect for the new object pool repository. For example, the object pool repository is assigned to storages 3, 4, and 5. Note that these assignments do not entirely match the upstream repository's.
1. The forked copy of the repository gets created with the `CreateFork()` RPC and is also assigned to three randomly-selected storages. For example, the fork repository gets assigned storages 1, 3, and 5. These assignments also do not entirely match the upstream and object pool repository's storage assignments.
1. Both the upstream and fork repositories are linked to the object pool via separate invocations of `LinkRepositoryToObjectPool()`. For this RPC to succeed the object pool must exist on the same storage as the repository that is linking to it. The upstream repository fails to link on storages 1 and 2. The fork repository fails to link on storage 2. The `LinkRepositoryToObjectPool()` RPC is not transactional so a single failure of the RPC on any of the storages results in an error being proxied back to the client. Therefore, in this scenario `LinkRepositoryToObjectPool()` on both the upstream and fork repository always result in an error response.

To fix this problem, we must ensure Praefect always routes `CreateObjectPool()` and `CreateFork()` RPC requests to the same set of storages as the upstream repository. This ensures that these repositories always have the required object pool repository available so that linking to them can succeed.

The main downside of this is that repositories in an object deduplication network are pinned to the same set of storages. This could unevenly stress individual storages as an object deduplication network grows larger. In the future this can be avoided altogether when Praefect has the ability to create object pools on a storage where it is required but not already present.

Repository replication

The `ReplicateRepository()` RPC is not aware of object pools and only replicates from the source repository. This means that replication of a source repository linked to an object pool repository results in a target repository with no Git alternates file and consequently no deduplication of objects.

The `ReplicateRepository()` RPC has two main uses:

- Storage moves performed in the GitLab API rely on the `ReplicateRepository()` RPC to replicate repositories from one storage to another. Since this RPC is

currently not object pool aware, the resulting replica on the target storage does not replicate the Git alternates file from the source repository or recreate any object pools. Instead, the replica is always a complete self-contained copy of the source repository. Consequently, the object pool relationship for the repository project in Rails is also removed. When moving repositories in an object deduplication network from one storage to another, the replicated repositories can result in increased storage usage because there is no longer any deduplication of objects on the target storage.

- When a repository replica becomes outdated in Praefect, the `ReplicateRepository()` RPC is internally used by Praefect replication jobs to replicate over the out-of-date replica from an up-to-date replica. Replication jobs are queued by the Praefect replication manager when replicas become outdated. Though the `ReplicateRepository()` RPC is not aware of object pools, the replication job checks if the source repository is linked to an object pool. If the source repository is linked, the job recreates the corresponding Git alternates file on the target repository. However, it is currently possible for an object pool to not exist on the same storage as the replica. When this happens, replication always fails because the replica is unable to link to the non-existent object pool. This means it is possible for replicas to remain outdated permanently.

Object pools required by the source repository should be replicated to the target storage along with the repository during the `ReplicateRepository()` RPC. This preserves object deduplication for repositories in an object deduplication network. Because storage moves performed in the GitLab API remove any object pool relationships, recreating object pools on the target storage results in orphaned object pools. This new object pool replication behavior of the `ReplicateRepository()` RPC should be controlled by the client to prevent breaking changes. Object pool replication for storage moves can be enabled once either:

- The Rails side is updated to preserve the object pool relationship.
- The object pool lifecycle is managed within Gitaly.

When it comes to replication of object pools, there are scenarios Praefect needs to be capable of handling. Special consideration must be made in these cases so Praefect can keep track of all the repositories it manages in its PostgreSQL database to ensure they stay up to date.

- Replication of an external source repository that is linked to an object pool to Gitaly Cluster can result in the target virtual storage's Praefect needing to create a new object pool repository. To handle this, it needs to be known if the source repository is using an object pool. From there it can be checked if Praefect has an entry for the object pool repository in its repositories database table, and if not, create one. Next, Praefect storage assignments for the object pool need to be generated and persisted in the `repositoryassignments` database table.
- It can not be guaranteed that the target repository storage in Praefect already contains the required object pool. Thus an individual storage may need to have an object pool assigned to it. This new assignment must also be

tracked by the object pool repository in Praefect. To handle this, Praefect has to detect when a target storage does not contain the required object pool and persist the new storage assignment in the repositoryassignments database table.

Problems with the design

As mentioned before, object pools are not a perfect solution. This section goes over the most important issues.

Complexity of lifecycle management

Even though the lifecycle of object pools becomes easier to handle once it is fully owned by Gitaly, it is still complex and needs to be considered in many ways. Handling object pools in combination with their repositories is not an atomic operation as any action by necessity spans over at least two different resources.

Performance issues

As object pools deduplicate objects, the end result is that object pool members never have the full closure of objects in a single packfile. This is not typically an issue for the primary object pool member, which by definition cannot diverge from the object pool's contents. But secondary object pool members can and often will diverge from the original contents of the upstream repository.

This leads to two different sets of reachable objects in secondary object pool members. Unfortunately, due to limitations in Git itself, this precludes the use of a subset of optimizations:

- Packfiles cannot be reused as efficiently when serving fetches to serve already-deltified objects. This requires Git to recompute deltas on the fly for object pool members which have diverged from object pools.
- Packfile bitmaps can only exist in object pools as it is not possible nor easily feasible for these bitmaps to cover multiple object databases. This requires Git to traverse larger parts of the object graph for many operations and especially when serving fetches.

Dependent writes across repositories

The design of object pools introduces significant complexity into the Raft world where we use a write-ahead log for all changes to repositories. In the ideal case, a Raft-based design would only need to care about the write-ahead log of a single repository when considering requests. But with object pools, we are forced to consider both reads and writes for a pooled repository to be dependent on all writes in its object pool having been applied.

Alternative Solutions

The proposed solution is not obviously the best choice as it has issues both with complexity (management of the lifecycle) and performance (inefficiently served fetches for pool members).

This section explores alternatives to object pools and why they have not been chosen as the new target architecture.

Stop using object pools altogether

An obvious way to avoid all of the complexity is to stop using object pools altogether. While it is charming from an engineering point of view as we can significantly simplify the architecture, it is not a viable approach from the product perspective as it would mean that we cannot support efficient forking workflows.

Primary repository as object pool

Instead of creating an explicit object pool repository, we could just use the upstream repository as an alternate object database of all forks. This avoids a lot of complexity around managing the lifetime of the object pool, at least superficially. Furthermore, it circumvents the issue of how to update object pools as it will always match the contents of the upstream repository.

It has a number of downsides though:

- Repositories can now have different states, where some of the repositories are allowed to prune objects and others aren't. This introduces a source of uncertainty and makes it easy to accidentally delete objects in a repository and thus corrupt its forks.
- When upstream repositories go private we must stop updating objects which are supposed to be deduplicated across members of the fork network. This means that we would ultimately still be forced to create object pools once this happens in order to freeze the set of deduplicated objects at the point in time where the repository goes private.
- Deleting repositories becomes more complex as we need to take into account whether a repository is linked to by forks.

Reference namespaces

With `gitnamespaces(7)`, Git provides a mechanism to partition references into different sets of namespaces. This allows us to serve all forks from a single repository that contains all objects.

One neat property is that we have the global view of objects referenced by all forks together in a single object database. We can thus easily perform shared housekeeping across all forks at once, including deletion of objects that are not used by any of the forks anymore. Regarding objects, this is likely to be the most efficient solution we could potentially aim for.

There are again some downsides though:

- Calculating usage quotas must by necessity use actual reachability of objects into account, which is expensive to compute. This is not a showstopper, but something to keep in mind.
- One stated requirement is that it must not be possible to make objects reachable in other repositories from forks. This property could theoretically be enforced by only allowing access to reachable objects. That way an object can only be accessed through virtual repository if the object is reachable from its references. Reachability checks are too compute heavy for this to be practical.
- Even though references are partitioned, large fork networks would still easily end up with multiple millions of references. It is unclear what the impact on performance would be.
- The blast radius for any repository-level attacks significantly increases as you would not only impact your own repository, but also all forks.
- Custom hooks would have to be isolated for each of the virtual repositories. Since the execution of Git hooks is controlled it should be possible to handle this for each of the namespaces.

Filesystem-based deduplication

The idea of deduplicating objects on the filesystem level was floating around at several points in time. While it would be nice if we could shift the burden of this to another component, it is likely not easy to implement due to the nature of how Git works.

The most important contributing factor to repository sizes are Git objects. While it would be possible to store the objects in their loose representation and thus deduplicate on that level, this is infeasible:

- Git would not be able to deltify objects, which is an extremely important mechanism to reduce on-disk size. It is unlikely that the size reduction caused by deduplication would outweigh the size reduction gained from the deltification mechanism.
- Loose objects are significantly less efficient when accessing the repository.
- Serving fetches requires us to send a packfile to the client. Usually, Git is able to reuse large parts of already-existing packfiles, which significantly reduces the computational overhead.

Deduplicating on the loose-object level is thus infeasible.

The other unit that one could try to deduplicate is packfiles. But packfiles are not deterministically generated by Git and will furthermore be different once repositories start to diverge from each other. So packfiles are not a natural

fit for filesystem-level deduplication either.

An alternative could be to use hard links of packfiles across repositories. This would cause us to duplicate storage space whenever any repository decides to perform a repack of objects and would thus be unpredictable and hard to manage.

Custom object backend

In theory, it would be possible to implement a custom object backend that allows us to store objects in such a way that we can deduplicate them across forks. There are several technical hurdles though that keep us from doing so without significant upstream investments:

- Git is not currently designed to have different backends for objects. Accesses to files part of the object database are littered across the code base with no abstraction level. This is in contrast to the reference database, which has at least some level of abstraction.
- Implementing a custom object backend would likely necessitate a fork of the Git project. Even if we had the resources to do so, it would introduce a major risk factor due to potential incompatibilities with upstream changes. It would become impossible to use vanilla Git, which is often a requirement that exists in the context of Linux distributions that package GitLab.

Both the initial and the operational risk of ongoing maintenance are too high to really justify this approach for now. We might revisit this approach in the future.

SECTION: engineering/architecture/design-documents/object_storage/_index.md

```
<!-- vale gitlab.FutureTense = NO -->
{{< engineering/design-document-header >}}
```

Abstract

GitLab stores three classes of user data: database records, Git repositories, and user-uploaded files (which are referred to as file storage throughout the blueprint).

The user and contributor experience for our file storage has room for significant improvement:

- Initial GitLab setup experience requires creation and setup of 13 buckets, instead of just 1.
- Features using file storage require contributors to think about both local storage and object storage, which leads to friction and complexity. This often results in broken features and security issues.
- Contributors who work on file storage often also have to write code

for Workhorse, Omnibus, and cloud native GitLab (CNG).

Problem definition

Object storage is a fundamental component of GitLab, providing the underlying implementation for shared, distributed, highly-available (HA) file storage.

Over time, we have built support for object storage across the application, solving specific problems in a multitude of iterations.

This has led to increased complexity across the board, from development (new features and bug fixes) to installation:

- New GitLab installations require the creation and configuration of several object storage buckets instead of just one, as each group of features requires its own. This has an impact on the installation experience and new feature adoption, and takes us further away from boring solutions.
- The release of cloud native GitLab required the removal of NFS shared storage and the development of direct upload, a feature that was expanded, milestone after milestone, to several type of uploads, but never enabled globally.
- Today, GitLab supports both local storage and object storage. Local storage only works on single box installations or with a NFS, which we no longer recommend to our users and is no longer in use on GitLab.com.
- Understanding all the moving parts and the flow is extremely complicated: we have CarrierWave, Fog, Go S3/Azure SDKs, all being used, and that complicates testing as well.
- Fog and CarrierWave are not maintained to the level of the native SDKs (for example, AWS S3 SDK), so we have to maintain or monkey patch those tools to support requested customer features (for example, issue 242245) that would normally be "free".
- In many cases, we copy around object storage files needlessly (for example, issue 285597).
Large files (for example, LFS and packages) are slow to finalize or don't work at all as a result.

Improvements over the current situation

The following is a brief description of the main directions we can take to remove the pain points affecting our object storage implementation.

This is also available as a YouTube video recorded for the Object Storage Working Group.

Simplify GitLab architecture by shipping MinIO

In the beginning, object storage support was a Premium feature, not part of our CE distribution. Because of that, we had to support both local storage and object storage.

With local storage, there is the assumption of a shared storage between components. This can be achieved by having a single box installation, without HA, or with a NFS, which we no longer recommend.

We have a testing gap on object storage. It also requires Workhorse and MinIO, which are not present in our pipelines, so too much is replaced by a mock implementation. Furthermore, the presence of a shared disk, both in CI and in local development, often hides broken implementations until we deploy on an HA environment.

One consideration we can take is to investigate shipping MinIO as part of the product. This could reduce the differences between a cloud and a local installation, standardizing our file storage on a single technology.

The removal of local disk operations would reduce the complexity of development as well as mitigate several security attack vectors as we no longer write user-provided data on the local storage.

It would also reduce human errors as we will always run a local object storage in development mode and any local file disk access should raise a red flag during the merge request review.

This effort is described in this epic.

Before considering any specific third-party technology, the open source software licensing implications should be considered. As of 23 April 2021, MinIO is subject to the AGPL v3 license. GitLab Legal must be consulted before any decision is taken to ship MinIO as proposed in this blueprint.

Enable direct upload by default on every upload

Because every group of features requires its own bucket, we don't have direct upload enabled everywhere. Contributing a new upload requires coding it in both Ruby on Rails and Go.

Implementing a new feature that does not have a dedicated bucket requires the developer to also create a merge request in Omnibus and CNG, as well as coordinate with SREs to configure the new bucket for our own environments.

This also slows down feature adoptions, because our users need to reconfigure GitLab and prepare a new bucket in their infrastructure. It also makes the initial installation more complex feature after feature.

Implementing a direct upload by default, with a consolidated object storage configuration will reduce the number of merge requests needed to ship a new feature from four to only one. It will also remove the need for SRE intervention as the bucket will always be the same.

This will simplify our development and review processes, as well as the GitLab configuration file. And every user will immediately have access to new features without infrastructure chores.

Simplify object storage code

Our implementation is built on top of a 3rd-party framework where every object storage client is a 3rd-party library. Unfortunately some of them are unmaintained.

We have customers who cannot push 5 GB Git LFS objects, but with such a vital feature implemented in 3rd-party libraries we are slowed down in fixing it, and we also rely on external maintainers to merge and release fixes.

Before the introduction of direct upload, using the CarrierWave library, "a gem that provides a simple and extremely flexible way to upload files from Ruby applications.", was the boring solution. However this is no longer our use-case, as we upload files from Workhorse, and we had to patch CarrierWave's internals to support direct upload.

A brief proposal covering CarrierWave removal and a new streamlined internal upload API is described in this issue comment.

Ideally, we wouldn't need to duplicate object storage clients in Go and Ruby. By removing CarrierWave, we can make use of the officially supported native clients when the provider S3 compatibility level is not sufficient.

Iterations

In this section we list some possible iterations. This is not intended to be the final roadmap, but is a conversation started for the Object Storage Working Group.

1. Create a new catchall bucket and a unified internal API for authorization without CarrierWave.
1. Ship MinIO with Omnibus (CNG images already include it).
1. Expand GitLab-QA to cover all the supported configurations.
1. Deprecate local disk access.
1. Deprecate configurations with multiple buckets.
1. Implement a bucket-to-bucket migration.

1. Migrate the current CarrierWave uploads to the new implementation.
1. On the next major release: Remove support for local disk access and configurations with multiple buckets.

Benefits of the current iteration plan

The current plan is designed to provide tangible benefits from the first step.

With the introduction of the catchall bucket, every upload currently not subject to direct upload will get its benefits, and new features could be shipped with a single merge request.

Shipping MinIO with Omnibus will allow us to default new installations to object storage, and Omnibus could take care of creating buckets. This will simplify HA installation outside of Kubernetes.

Then we can migrate each CarrierWave uploader to the new implementation, up to a point where GitLab installation will only require one bucket.

Additional reading materials

- Uploads development guide.
- Speed up the monolith, building a smart reverse proxy in Go: a presentation explaining a bit of workhorse history and the challenge we faced in releasing the first cloud-native installation.
- Object Storage improvements epic.
- We are moving to GraphQL API, but we do not support direct upload.

SECTION: engineering/architecture/design-documents/observability_for_self_managed/_index.md

{{< engineering/design-document-header >}}

Summary

GitLab observability will first be available to self-managed instances through Cloud Connector so self-managed users can take advantage of GitLab Observability without the need to manage an observability platform. This document outlines the architecture.

In addition to making GitLab observability available to self-managed instances through Cloud Connector, GitLab observability available in GitLab.com will also realign architecture to utilize Cloud Connector.

The following architectural overview can be applied to both self-managed GitLab instances and .com GitLab instances.

Goals

- Offer seamless Observability features to self-managed users without requiring them to manage a scalable and reliable observability system.
- Move Observability APIs to GitLab's project resource API.
- Maintain API consistency between .com and self-managed GitLab

Architecture

```

plantuml
@startuml
node "GitLab Inc. Infrastructure" {
  package "Private Services" {
    [GitLab Observability Backend (GOB)] as GOB yellow
  }

  package "Public Services" {
    [GitLab (SaaS)] as SAAS
    [Cloud Connector gateway] as CC
    [Customers Portal] as CDOT
  }
}

```

```

node "Customer Infrastructure" {
  [OpenTelemetry SDK/Collector] as OT
  [GitLab (Self-Managed)] as SM
  [Sidekiq] as SK
}

```

```

OT --> SM : "send"
SAAS -[blue]-> CC : "access GOB via CC"
CC -[blue]-> GOB : "access GOB"
GOB --> SAAS : "fetch JWKS"
GOB --> CDOT : "fetch JWKS"

```

```

SM -[blue]-> CC : "access GOB via CC"
SK --> CDOT : "sync CC access token"

```

note bottom of CC

Makes routing decisions:

```
cloud.gitlab.com/observe/ ---> GOB
```

...

end note

note top of GOB

- Use JWKS from SaaS to auth SaaS requests

- Use JWKS from Customers Portal to auth
Self-Managed GitLab requests

...

end note

@enduml

GitLab Observability Backend (GOB) deployments will be managed by GitLab Inc. as part of the GitLab Inc. infrastructure. GitLab.com and all self-managed GitLab instances will initially use the same GOB backend that exists today. However, in the future, we could also deploy regional GOB instances, or even customer-specific GOB instances if the need arises, and route requests to the desired GOB instance.

Blue arrows highlight the request path from both GitLab SaaS and GitLab self-managed instances to GOB. All requests pass through the Cloud Connector Gateway public service and on to the private GOB service. GitLab SaaS cells will all have access to the public Cloud Connector Gateway.

The Cloud Connector Gateway will be a single entry point Load balancer.

Detailed request architecture is outlined in the next section.

Walkthrough of Observability Requests

The following two diagrams highlight the request flow for GitLab.com customers and for self-managed GitLab customers.

A couple small differences exist between the two:

1. The JWT sent to GOB in GitLab.com requests is created in Rails, whereas the JWT or otherwise referred to as IJWT in the self-managed flow originates from CustomersDot.
1. There is no CustomersDot JWT sync in the GitLab.com flow.

Observability offered in GitLab.com

```
mermaid
sequenceDiagram
    autonumber
    participant U as GitLab.com user
    participant SDK as OpenTelemetry SDK/Collector
    participant UI as GitLab.com (UI)
    participant W as GitLab.com (WorkHorse)
    participant R as GitLab.com (Rails)
    participant GOB as GOB SaaS behind cloud.gitlab.com (GitLab Observability Backend)
```

Note over U,GOB: Configure SDK/Collector

U->>.comR: create Group/Project PAT

.comR-->>U: PAT

U->>SDK: configure with PAT & Project endpoint

Note over U,GOB: Sending OpenTelemetry Data

loop Requests issued by SDK/Collector

SDK->>.comW: send opentelemetry data with PAT

.comW->>.comR: preAuthHandler without body

```
.comR->>.comR: auth with PAT
.comR-->>.comW: metadata + JWT
.comW->>GOB: send data with JWT + metadata headers
GOB->>GOB: validate JWT & store data
GOB-->>.comW: response
.comW-->>SDK: response
end
```

Note over U,GOB: GitLab UI

```
U->>.comUI: view
.comUI->>.comW: request with cookies
.comW->>.comR: preAuthHandler forward cookies
.comR->>.comR: auth with cookies
.comR-->>.comW: metadata + JWT
.comW->>GOB: forward with JWT + metadata headers
GOB->>GOB: validate JWT & return results
GOB-->>.comW: response
.comW-->>.comUI: response
```

Note over U,GOB: Direct to API

```
U->>.comR: create Group/Project PAT
.comR-->>U: PAT
U->>.comW: request with PAT
.comW->>.comR: preAuthHandler
.comR->>.comR: auth with PAT
.comR-->>.comW: metadata + JWT
.comW->>GOB: forward with JWT + metadata headers
GOB->>GOB: validate JWT & return results
GOB-->>.comW: response
.comW-->>U: response
```

Observability offered in Self-Managed GitLab

mermaid

sequenceDiagram

autonumber

participant U as SM user

participant SDK as OpenTelemetry
 SDK/Collector

participant SMUI as SM GitLab (UI)

participant SMW as SM GitLab (WorkHorse)

participant SMR as SM GitLab (Rails)

participant CD as CustomersDot

participant GOB as GOB SaaS behind cloud.gitlab.com
(GitLab Observability Backend)

Note over U,GOB: instance-to-instance auth

loop chron job

SMR->>CD: sync Cloud Connector access data

Note over CD,SMR: Includes IJWT scoped to eligible services

CD-->>SMR: access data + IJWT
SMR-->>SMR: store access data + IJWT
end
Note over U,GOB: Configure SDK/Collector
U->>SMR: create Group/Project PAT
SMR-->>U: PAT
U->>SDK: configure with PAT & Project endpoint

Note over U,GOB: Sending OpenTelemetry Data
loop Requests issued by SDK/Collector
SDK->>SMW: send opentelemetry data with PAT
SMW->>SMR: preAuthHandler without body
SMR->>SMR: auth with PAT
SMR-->>SMW: metadata + IJWT
SMW->>GOB: send data with IJWT + metadata headers
GOB->>GOB: validate IJWT & store data
GOB-->>SMW: response
SMW-->>SDK: response
end

Note over U,GOB: GitLab UI
U->>SMUI: view
SMUI->>SMW: request with cookies
SMW->>SMR: preAuthHandler forward cookies
SMR->>SMR: auth with cookies
SMR-->>SMW: metadata + IJWT
SMW->>GOB: forward with IJWT + metadata headers
GOB->>GOB: validate IJWT
GOB-->>SMW: response
SMW-->>SMUI: response

Note over U,GOB: Direct to API
U->>SMR: create Group/Project PAT
SMR-->>U: PAT
U->>SMW: request with PAT
SMW->>SMR: preAuthHandler
SMR->>SMR: auth with PAT
SMR-->>SMW: metadata + IJWT
SMW->>GOB: forward with IJWT + metadata headers
GOB->>GOB: validate IJWT
GOB-->>SMW: response
SMW-->>U: response

Performance

It is critically important to keep the body of any observability requests from burdening Rails/Puma. All preAuthHandlers in Workhorse will ensure the body is not forwarded to Rails and will only be forwarded to GOB when auth is successful.

If we consider the daily data transmitted and stored concerning our observability of GitLab.com, and we extrapolate that across our self-managed customers who might have equally demanding needs, we can get an idea of how much data will pass through Workhorse and Cloud Connector. GitLab.com produces 150M+ metrics series, sampled every 30-60 seconds, and 18-22 TB of logs per day.

We could assume that any Ultimate tier root-level namespace on GitLab.com or any Ultimate tier self-managed instance could send a similar magnitude of data through cloud.GitLab.com.

To get an idea of what this equals to in requests/second and bytes/second, we walk through the following basic example.

Assumptions:

- Compression: use this chart that illustrates compression ratios for observability data types
- An example metric is 396 Bytes raw and translates to a mdmetricrequest in the compression table linked above:
 - {name="cluster:namespace:podcpu:active:kubepodcontainerresourcelimits", cluster="play-db-cluster", container="k6-grpc", instance="10.4.3.6:8080", job="integrations/Kubernetes/kube-state-metrics", namespace="play-backends", node="gke-raintank-dev-pla-raintank-dev-pla-3cd3aafc-hijt", pod="k6-grpc-5c74969fdc-6n2bn", resource="cpu", uid="56e073b7-ca28-4cf4-b3da-83dc97763bef", unit="core"}
- Trace volume can easily exceed log volume, with similar context attached to each span as would be found in a log line. Tracing is more similar to debug logging, so sampling is used to reduce the overall volume to reasonable levels. Let's assume for argument's sake that tracing volume after sampling is equal to logging volume.
- For compression ratios, assume mdlogrequest, mdtracerequest and mdmetricrequest

Single large Ultimate tier Organization (via GitLab.com or self-managed):

- Metrics volume: 150M active metrics series sampled every 30 seconds
 - Samples/sec = $150M / 30 = 5M$
 - Bytes/sec = $5M \times 396B / 2.21 \text{ compression ratio} = 896MB/s$
- Logs volume: 18TB/day
 - Bytes/sec uncompressed = $18e+12 / (24 \times 60 \times 60) = 208MB/s$
 - Bytes/sec compressed = $208 / 1.35 \text{ compression ratio} = 154Mb/s$
- Traces volume: from assumptions above, assume to be the same as log volume
 - Bytes/sec compressed = $208 / 1.57 \text{ compression ratio} = 132Mb/s$

Treating this as an upper bound assumption that the total possible demand from an Ultimate tier customer could be $896 + 154 + 132 = 1.2GB/s$ for data ingestion.

Data ingestion demands are always magnitudes higher than reading back observability data, so we can disregard read load for the sake of this back-of-napkin math.

It is also useful to consider the average requests per second that deliver the 1.2GB/s. This will allow us to infer the number of preAuthHandler requests against Rails.

The OpenTelemetry Batch Processor is the recommended way to send metrics, logs, and traces in

configurable-sized batches. The default batch size is 8192 events.

If we take the average compressed size of mdlogrequest, mdtracerequest and mdmetricrequest from the Opentelemetry compression comparison and the Metric, Log, Trace Volume outlined above we can compute the average request/second using the default batch size.

- Metrics req/s:

- 896MB per sec / 145 compressed bytes per event = 6.2M events/sec
- 6.2M events per sec / 8192 events per batch = 760 reqs/sec

- Logs req/s:

- 154MB per sec / 268 compressed bytes per event = 0.57M events/sec
- 0.57M events per sec / 8192 events per batch = 70 reqs/sec

- Traces req/s:

- 132MB per sec / 288 compressed bytes per event = 0.46M events/sec
- 0.46M events per sec / 8192 events per batch = 56 reqs/sec

Total requests per second using default batching is 890 requests/second for each large customer with 150M active metrics series, 18TB of daily logs, and 18TB of daily traces.

Observability for GitLab.com

For each customer of this magnitude on GitLab.com, there would be a demand of 1.2GB/s through Workhorse.

To maximize performance and GitLab service availability, a dedicated web fleet should be created for observability just like the recent addition for code suggestions. All incoming request paths starting with /api/v4/projects/:id/observability/ can be routed to the dedicated observability web fleet in .com.

This mitigates the risk of observability requests saturating the main GitLab web fleet and allows for horizontal autoscaling of the observability fleet based on demand.

Observability for Self-Managed

The self-managed Workhorse throughput for this magnitude of data would be 1.2GB/s. Self-managed customers will have access to the appropriate configuration in our reference architecture documentation.

Authentication & Authorization

As outlined in the request flow, there are two main request legs for any interaction with GOB.

1. User/Client to their GitLab instance
1. GitLab instance to GOB

For all leg 1 requests, authentication and authorization will be enforced by leveraging a preAuthorizeHandler in Workhorse. This handler will call an internal Rails API to perform both authN and authZ using either the supplied PAT or browser cookies. The internal Rails APIs

will first authenticate the request to ensure it was issued by Workhorse using the Workhorse/Rails JWT and shared signing secret. If successful, the endpoint will then perform the appropriate auth checks against the supplied PAT/browser cookies. If successful, Rails will return in the response back to Workhorse, all the headers that need to be sent to GOB including the IJWT (if instance is self-managed) and any metadata, in a similar manner to Workhorse/Rails websocket auth-data.

Workhorse will then forward the request to GOB with either the IJWT (self-managed) or the Workhorse JWT (SaaS). GOB will then verify the JWT to authenticate the request.

Both Rails and Workhorse use the same secret to sign JWTs so the JWKS available at the `jwksuri` found at <https://gitlab.com/.well-known/openid-configuration> can be used to verify the identity of the JWT created from Rails or Workhorse, meaning requests to GOB could also be sent from Rails. For the IJWTs, the JWKS can be retrieved in accordance with this.

Enabling

Self-Managed users will enable the use of Cloud Connected GitLab Observability through the Customers Portal. All relevant licenses and subscription info will then be encoded into the IJWT which Workhorse, Rails, and GOB will use to manage access and enforce limits and quotas.

Rate-limits and Quota Management

GitLab Observability Backend already has rate limiting on its API gateway to prevent denial of service attacks.

In addition to these existing rate limits, we will also add rate limiting to Workhorse/Rails in the same idiomatic manner to which other GitLab APIs manage rate limits to ensure rate limits are enforced at the edge, mitigating any DOS attacks at the edge before calling on more downstream services.

GitLab Observability Quota management is enforced in GOB. GOB configures compute quota for each top-level namespace, depending on the customer's license. In addition that, it will enforce storage quota appropriately based on the customer license by tracking storage and purging old data to make space for new data, ensuring storage is always kept below an upper bound for each top-level namespace. Eventually, this quota management will be mapped to the GitLab Organization construct instead of top-level namespace.

To ensure rate limiting and quota management in the GOB service is enforced correctly, GOB will use the IJWT token to extract relevant information about the customer license. Metadata headers will also be sent as part of the request leg from Workhorse to GOB that provides any additional information required by GOB to fulfill requests and enforce quotas and limits.

APIs

All observability APIs will proxy through Workhorse and will live under GitLabs project resource.

HTTP will be supported initially and gRPC will be added at a later date.

System Monitoring

GitLab Observability Backend already has system monitoring in place that plugs into GitLabs centralized metrics and logging system. Additional metrics will also be captured in Workhorse and in Rails to provide visibility and alerting on the user-facing API.

SECTION: engineering/architecture/design-documents/observability_logging/_index.md

{{< engineering/design-document-header >}}

Summary

This design document outlines a system for storing and querying logs which will be a part of GitLab Observability Backend (GOB), together with tracing and metrics.

At its core the system is leveraging OpenTelemetry logging specification for data ingestion and ClickHouse database for storage.

The users will interact with the data through GitLab UI.

The system itself is multi-tenant and offers our users a way to store their application logs, query them, and in future iterations correlate with other observability signals (traces, errors, metrics, etc...).

Motivation

After tracing and metrics, logging is the last observability signal that we need to support to be able to provide our users with a fully-fledged observability solution.

One could argue that logging itself is also the most important observability signal because it is so widespread.

It predates metrics and tracing in the history of application observability and is usually implemented as one of the first things during development.

Without logging support, it would be very hard if not impossible to fully understand for our users the performance and operation of the applications developed by them with the help of our platform.

Goals

- multi-tenant: each user and their data should be isolated from others that are using the platform.

- Users may query only the data that they have sent to the platform.

- follows OpenTelemetry standards: logs ingestion should follow the OpenTelemetry protocol.

- Apart from being able to re-use the tooling and know-how that was already developed for OpenTelemetry protocol, we will not have to reinvent the wheel when it comes to wire protocol and data storage format.

- uses ClickHouse as a data storage backend: ClickHouse has become the go-to solution for observability data at GitLab for a plethora of reasons.

Our tracing and metrics solutions already use it, so logging should be consistent with it and not introduce new dependencies.

- Users can query their data using reasonably complex queries: storing logs by itself will not bring much value to our users.

Non-Goals

- complex query support and logs analytics - at least in the first iteration we do not plan to support complex queries, in particular GROUP BY queries that users may want to use for quantitative logs analytics.

Supporting it is not trivial and requires some more research and work in the area of query language syntax.

- advanced data retention - logs differ from traces and metrics concerning legal requirements. Authorities may request logs stored by us as part of e.g. ongoing investigations.

In the initial iteration, we need to caution our users that our system is not ready for that and they need a secondary system for now if they intend to store e.g. access logs.

We will need more work around logs/data integrity and long-term storage policies to handle this use case.

- data deletion - apart from the case where the data simply expires after a predefined storage period, we do not plan to support deleting individual logs by users.

This is left for later iterations.

- linking logs to traces - we do not intend to support linking logs to traces in the first iteration, at least not in the UI.

- logs sampling - for traces we expect users to sample their data before sending it to us while we focus only on enforcing the limits/quotas.

Logs should follow this pattern.

The log sampling implementation seems immature as well - a log sampler is implemented in OTEL Collector, but it is not clear if it can work together with traces sampling, and there is no official specification (issue, pull request).

Proposal

The architecture of logs ingestion follows the patterns outlined in the tracing and metrics proposals:

We re-use the components that were introduced by these proposals, so there are not going to be any new services added.

Each top-level GitLab namespace has its own OTEL collector to which ingestion requests are directed by the cluster-wide Ingress.

On the other hand, there is a single, cluster-wide query service that handles queries from users.

The query service is tenant-aware.

Rate-limiting of the user requests is done at the Ingress level.

The cluster-wide Ingress is currently done using Traefik, and it is shared with all other services in the cluster.

Ingestion path

We receive Log objects from customers in the JSON format over HTTP.

The request arrives at the cluster-wide Ingress which routes the request to the appropriate OTEL collector.

The collector then processes this request and executes INSERT statements against Clickhouse.

Read path

GOB exposes an HTTP/JSON API that e.g. GitLab UI uses to query and then render logs.

The cluster-wide Ingress is routing the requests to the query service which in turn parses the API request and executes an SQL query against ClickHouse.

The results are then formatted into JSON response and sent back to the client.

Design and implementation details

Legacy code

Handling logging signals is heavily influenced by the large amount of legacy code that needs to be supported, contrary to trace and metric signals.

For metrics and tracing, OpenTelemetry specification defines new APIs and SDKs that can be leveraged.

With logs, OpenTelemetry acts more like a bridge and enables legacy libraries/code to send their data to us.

Users may create Log signals from plain log files using filelogreceiver or fluentd.

Existing log libraries may use Log Bridge API to emit logs using OTEL protocol.

In time the ecosystem will most probably develop and the number of options will grow.

The assumption is made that how logs are ingested is up to the user.

Hence we expose only an HTTP endpoint that accepts logs in OTEL format and assume that logs are already properly parsed and forma